Day 1 – Problem Solving Techniques and Data Structures – I

Set 1 – Retail Order Desk

Dataset: orders.csv

OrderId	Customerid	amount	CreatedAt
129525	899	3737.80	2026-10-02 10:15
131065	714	642.03	2026-10-03 11:20
128729	645	710.84	2026-10-02 12:30
124602	6722	1780.85	2026-10-04 09:45
105487	310	3544.32	2026-10-07 14:10

Task 1: Linear Search and Binary Search

```
def linear_search(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i
    return -1


def binary_search(a, x):
    low, high = 0, len(a) - 1

    while low <= high:
        mid = (low + high) // 2

        if a[mid] == x:
            return mid
        elif a[mid] < x:
            low = mid + 1
```

```
        else:
            high = mid - 1

    return -1

x = 105487
sorted_data = sorted(data)

start = time.perf_counter()
print("Linear Search:", linear_search(data, x))
print("Linear Time:", time.perf_counter() - start)

start = time.perf_counter()
print("Binary Search:", binary_search(sorted_data, x))
print("Binary Time:", time.perf_counter() - start)
```

Task 2: Bubble Sort and Merge Sort

```
def bubble_sort(a):
    a = a[:]
    n = len(a)

    for i in range(n):
        for j in range(n - i - 1):
            if a[j] < a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
```

```
    return a

def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] >= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    return result + left[i:] + right[j:]

start = time.perf_counter()
print("Bubble Sort:", bubble_sort(amounts))
print("Bubble Time:", time.perf_counter() - start)
```

```
start = time.perf_counter()
print("Merge Sort:", merge_sort(amounts))
print("Merge Time:", time.perf_counter() - start)

start = time.perf_counter()
print("Built-in Sort:", sorted(amounts, reverse=True))
print("Built-in Time:", time.perf_counter() - start)
```

Task 3: Duplicate Order Detection Using Hash-Based Structure

```
seen = set()
count = 0
    key = (
        r["customerId"],
        r["amount"],
        r["createdAt"][:16]
    )

    if key in seen:
        count += 1
    else:
        seen.add(key)
```

```
print("Duplicate Orders:", count)
```

Task 4: Implementation of Stack and Queue

```
from collections import deque

class Stack:
    def __init__(self):
        self.s = []

    def push(self, x):
        self.s.append(x)

    def pop(self):
        return self.s.pop()

class Queue:
    def __init__(self):
        self.q = []

    def add(self, x):
        self.q.append(x)

    def remove(self):
        return self.q.pop(0)
```

```
st = Stack()
st.push("Edit 1")
st.push("Edit 2")

print("Undo:", st.pop())

q = Queue()
q.add(201)
q.add(202)

print("Dispatch:", q.remove())

d = deque([201, 202])
print("Built-in Dispatch:", d.popleft())
```